

Justificacion del uso de MongoDB en el Trabajo Practico

Trabajo Practico - LinkPro (clon de LinkedIn)

1. Contexto del trabajo practico

El TP consiste en construir una red social similar a LinkedIn integrando multiples motores de bases de datos para distintas funcionalidades. Segun la consigna, es obligatorio usar al menos una base de datos relacional (SQL), una documental (MongoDB) y una de grafos o clave-valor, justificando la eleccion de cada tecnologia segun las características propias de los datos y las operaciones requeridas.

El sistema LinkPro distribuye los datos en cuatro motores especializados:

- PostgreSQL/Supabase (SQL): usuarios, perfiles, empleos, postulaciones y grupos.
- MongoDB Atlas (documental): feed social - publicaciones, likes y comentarios.
- Cassandra (columnar distribuido): notificaciones y mensajes directos entre usuarios. [pendiente]
- Neo4j (grafos): conexiones entre usuarios y matching de habilidades. [pendiente]

Este documento justifica el uso de MongoDB para el modulo del feed social: publicaciones, likes y comentarios. Este modulo concentra el mayor volumen de escrituras y lecturas del sistema y tiene características que lo hacen inadecuado para un modelo relacional estricto.

2. Funcionalidades implementadas con MongoDB

El modulo del feed utiliza tres colecciones en MongoDB Atlas (base de datos: linkpro):

Coleccion	Equivalente SQL	Descripcion
posts	posts (tabla)	Publicaciones del feed con autor, contenido, media y tags
likes	likes (tabla)	Registro de 'Me gusta' por usuario y por post
comments	comments (tabla)	Comentarios de usuarios sobre publicaciones

Las colecciones likes y comentarios se disenán como colecciones separadas (no embebidas en posts) para permitir consultas eficientes por usuario, borrado individual y conteos independientes sin recargar el documento completo del post.

3. Por que MongoDB para el feed social

3.1 Documentos flexibles para contenido heterogeneo

Las publicaciones del feed tienen una estructura variable: algunas tienen solo texto, otras incluyen imagenes o videos, otras llevan etiquetas (tags), y el formato puede evolucionar a futuro sin requerir una migracion de esquema. En MongoDB, cada documento puede tener campos distintos sin romper la coleccion:

```
// Post solo con texto
{ _id: ObjectId, user_id: '...', content: 'Texto', tags: [], media: null, created_at: ISODate }
```

```
// Post con imagen y tags
{ _id: ObjectId, user_id: '...', content: 'Texto', tags: ['React','MongoDB'],
  media: { type: 'image', url: 'https://...' }, created_at: ISODate }
```

Con SQL, agregar el campo media o tags implicaria alterar la tabla (ALTER TABLE) y posiblemente crear tablas auxiliares como post_tags o post_media.

3.2 Alto volumen de escrituras - patron de uso del feed

El feed social es el modulo mas activo de cualquier red profesional. Los usuarios leen, publican, dan likes y comentan de forma continua y concurrente. MongoDB esta disenado para cargas de alta escritura con particionamiento horizontal (sharding), lo que lo hace mas adecuado para esta escala que una base relacional normalizada donde cada like o comentario implicaria bloqueos de fila.

- Cada like es un insert simple en la coleccion likes (no un UPDATE sobre el documento del post).
- Cada comentario es un insert en la coleccion comments con referencia al post_id.
- No hay locks de tabla ni transacciones complejas para las operaciones del feed.

3.3 Aggregation Pipeline: joins eficientes entre colecciones

Aunque MongoDB es una base documental, su Aggregation Pipeline permite realizar operaciones equivalentes a JOINS relacionales mediante el operador \$lookup. La consulta GET /posts utiliza este mecanismo para traer likes y comentarios junto a cada post en una sola operacion:

```
db.posts.aggregate([
  { $sort: { created_at: -1 } },
  { $limit: 50 },
  { $lookup: {
    from: 'likes',
    localField: '_id',
    foreignField: 'post_id',
    as: 'likes_list'
  }},
  { $lookup: {
    from: 'comments',
    localField: '_id',
    foreignField: 'post_id',
    pipeline: [{ $sort: { created_at: 1 } }],
    as: 'comments_list'
  }},
  { $addFields: {
    likes_count: { $size: '$likes_list' },
    comments_count: { $size: '$comments_list' },
    user_liked: { $in: [user_id, '$likes_list.user_id'] }
  }},
  { $project: { likes_list: 0 } }
```

1)

Este pipeline devuelve en una sola operacion: el post, la cantidad de likes, si el usuario actual dio like, la cantidad de comentarios y la lista de comentarios ordenada. Esto seria equivalente a multiples JOINS en SQL pero aprovechando la naturaleza orientada a documentos de MongoDB.

3.4 Colecciones separadas vs. embebido: disenio elegido y su justificacion

MongoDB permite dos enfoques para modelar likes y comentarios:

- EMBEBIDO: guardar los arrays de likes y comentarios dentro del documento del post. Rapido para leer, pero limita el tamaño del documento (maximo 16 MB), complica los borrados individuales y hace ineficiente contar likes de un usuario en todos sus posts.

- COLECCIONES SEPARADAS (enfoque elegido): likes y comments son colecciones independientes con un campo post_id que referencia al post. Permite indices, consultas por usuario, borrados atomicos y no tiene limite de tamaño.

Se eligio el enfoque de colecciones separadas por tres razones principales: (1) es una buena practica para datos que crecen sin limite (un post puede tener miles de comentarios), (2) permite consultas eficientes del tipo 'todos los posts que le gustaron a un usuario' sin recorrer todos los posts, y (3) la eliminacion de un like o comentario es un delete directo sin necesidad de modificar el documento del post.

3.5 Integracion con MongoDB Atlas

La aplicacion utiliza MongoDB Atlas, el servicio cloud de MongoDB. Esto permite:

- Base de datos en la nube sin necesidad de instalar ni configurar un servidor local.
- Conexion desde el backend Flask mediante una URI SRV (mongodb+srv://) con autenticacion.
- Panel de administracion web para visualizar colecciones, documentos e indices.
- Backups automaticos y alta disponibilidad con replica sets.

La conexion se configura en el archivo .env con las variables MONGO_URI y MONGO_DB, y se gestiona mediante el modulo db_mongo.py que inicializa el cliente pymongo con timeouts configurados para evitar bloqueos ante problemas de red.

```
# db_mongo.py
from pymongo import MongoClient

def get_db():
    client = MongoClient(MONGO_URI,
                        serverSelectionTimeoutMS=8000,
                        connectTimeoutMS=8000,
                        socketTimeoutMS=8000)
    return client[MONGO_DB]
```

4. Diseno de las colecciones

4.1 Coleccion: posts

```
{
  _id: ObjectId (autogenerado),
```

Justificacion del uso de MongoDB en el TP - LinkPro

```
user_id:      string    (referencia al usuario en SQL/Supabase),
author_name:  string,
author_surname: string,
author_photo_url: string | null,
content:      string    (texto de la publicacion),
media:        { type: 'image'|'video', url: string } | null,
tags:         [string] (etiquetas sin el # ),
created_at:   ISODate,
updated_at:   ISODate
}
```

4.2 Coleccion: likes

```
{
  _id:      ObjectId (autogenerado),
  post_id:  ObjectId (referencia a posts._id),
  user_id:  string    (referencia al usuario en SQL),
  created_at: ISODate
}
```

Indice recomendado: { post_id: 1, user_id: 1 } unique, para evitar likes duplicados y acelerar la busqueda.

4.3 Coleccion: comments

```
{
  _id:      ObjectId (autogenerado),
  post_id:  ObjectId (referencia a posts._id),
  user_id:  string    (referencia al usuario en SQL),
  author_name: string,
  author_surname: string,
  text:     string,
  created_at: ISODate
}
```

Indice recomendado: { post_id: 1, created_at: 1 } para recuperar comentarios de un post ordenados cronologicamente.

5. Endpoints de la API REST del feed (Blueprint posts_bp)

Metodo	Endpoint	Descripcion
GET	/posts?user_id=X	Lista los 50 posts mas recientes con likes y comentarios (aggregation)
POST	/posts	Crea una nueva publicacion en la coleccion posts
DELETE	/posts/<post_id>	Elimina el post y en cascada sus likes y comentarios
POST	/posts/<post_id>/like	Toggle de like: inserta o elimina en la coleccion likes
GET	/posts/<post_id>/comments	Lista comentarios de un post ordenados por fecha
POST	/posts/<post_id>/comments	Agrega un comentario en la coleccion comments
DELETE	/posts/<post_id>/comments/<comm_id>	Elimina un comentario por su _id

6. Queries MongoDB ejecutadas desde el backend

Las colecciones e indices de MongoDB se crean automaticamente al arrancar el servidor (db_mongo.py). Las operaciones de lectura y escritura se realizan mediante pymongo desde el blueprint posts_bp. A continuacion se detallan las operaciones por endpoint:

6.1 GET /posts - Listar publicaciones con likes y comentarios

Usa el Aggregation Pipeline con \$lookup para obtener en una sola consulta los posts junto con sus likes y comentarios:

```
db.posts.aggregate([
  { $sort: { created_at: -1 } },
  { $limit: 50 },
  { $lookup: {
    from: 'likes', localField: '_id',
    foreignField: 'post_id', as: 'likes_list'
  }},
  { $lookup: {
    from: 'comments', localField: '_id',
    foreignField: 'post_id',
    pipeline: [{ $sort: { created_at: 1 } }],
    as: 'comments_list'
  }},
  { $addFields: {
    likes_count: { $size: '$likes_list' },
    comments_count: { $size: '$comments_list' },
    user_liked: { $in: [user_id, '$likes_list.user_id'] }
  }},
  { $project: { likes_list: 0 } }
])
```

El resultado incluye: datos del post, likes_count, si el usuario actual dio like (user_liked), comments_count y la lista completa de comentarios ordenados cronologicamente.

6.2 POST /posts - Crear publicacion

```
db.posts.insert_one({
  user_id: user_id,
  author_name: name,
  author_surname: surname,
  author_photo_url: photo_url,
  content: content,
  media: { type, url } | None,
  tags: [lista de strings],
  created_at: datetime.utcnow(),
  updated_at: datetime.utcnow()
})
```

6.3 DELETE /posts/<post_id> - Eliminar post en cascada

MongoDB no tiene ON DELETE CASCADE, por lo que el backend elimina manualmente las tres colecciones:

```
db.posts.delete_one({ '_id': ObjectId(post_id) })
db.likes.delete_many({ 'post_id': ObjectId(post_id) })
db.comments.delete_many({ 'post_id': ObjectId(post_id) })
```

6.4 POST /posts/<post_id>/like - Toggle de like

Si el usuario ya dio like lo elimina, si no lo inserta (toggle). El indice unique previene duplicados:

```
# Verificar si ya existe el like:
db.likes.find_one({ 'post_id': ObjectId(post_id), 'user_id': user_id })

# Si existe -> eliminar:
db.likes.delete_one({ 'post_id': ObjectId(post_id), 'user_id': user_id })

# Si no existe -> insertar:
db.likes.insert_one({
    'post_id': ObjectId(post_id),
    'user_id': user_id,
    'created_at': datetime.utcnow()
})
```

6.5 GET /posts/<post_id>/comments - Listar comentarios

```
db.comments.find(
    { 'post_id': ObjectId(post_id) }
).sort('created_at', 1)
```

6.6 POST /posts/<post_id>/comments - Agregar comentario

```
db.comments.insert_one({
    'post_id': ObjectId(post_id),
    'user_id': user_id,
    'author_name': name,
    'author_surname': surname,
    'text': text,
    'created_at': datetime.utcnow()
})
```

6.7 DELETE /posts/<post_id>/comments/<comm_id> - Eliminar comentario

```
db.comments.delete_one({
    '_id': ObjectId(comm_id),
    'user_id': user_id # solo el autor puede borrar
})
```

6.8 Inicializacion automatica de colecciones e indices (db_mongo.py)

Justificacion del uso de MongoDB en el TP - LinkPro

Al arrancar el servidor, `get_db()` llama a `_init_collections()` que crea las colecciones e indices si no existen:

```
# posts
db.posts.create_index([('created_at', DESCENDING)], name='idx_posts_date')
db.posts.create_index([('user_id', ASC), ('created_at', DESC)], name='idx_posts_user_date')
db.posts.create_index([('tags', ASCENDING)], name='idx_posts_tags')

# likes
db.likes.create_index([('post_id', ASC), ('user_id', ASC)],
    unique=True, name='idx_likes_post_user')
db.likes.create_index([('user_id', ASCENDING)], name='idx_likes_user')

# comments
db.comments.create_index([('post_id', ASC), ('created_at', ASC)],
    name='idx_comments_post_date')
db.comments.create_index([('user_id', ASCENDING)], name='idx_comments_user')
```

7. Comparacion SQL vs MongoDB para el feed

Criterio	SQL (PostgreSQL)	MongoDB (elegido para feed)
Esquema	Fijo, requiere migraciones	Flexible, sin migraciones
Posts con media	Tabla auxiliar post_media	Campo media en el documento
Tags por post	Tabla auxiliar post_tags	Array tags en el documento
Alto volumen likes	Locks en UPDATE de contador	Insert atomico en coleccion likes
Joins	JOINS nativos con FK	\$lookup en aggregation pipeline
Escalabilidad	Vertical (mayor servidor)	Horizontal (sharding en Atlas)
Borrado en cascada	ON DELETE CASCADE en FK	Delete explicito en 3 colecciones
Indices	Indices sobre FK y campos	Indices compuestos por coleccion

8. Integracion en la arquitectura del sistema

La arquitectura del sistema separa claramente las responsabilidades de cada base de datos:

```
Frontend (React + TypeScript)
  |
  v
Backend Flask (Python)
  +-- /auth, /profile, /jobs, /groups --> Supabase (PostgreSQL)
  |                                     Usuarios, perfiles, empleos,
  |                                     postulaciones, grupos
  |
  +-- /posts --> MongoDB Atlas
  |               Posts, likes, comentarios (feed)
  |
```

Justificacion del uso de MongoDB en el TP - LinkPro

```
+-- /notifications, /messages    --> Cassandra [pendiente]
|                                Notificaciones, mensajes directos
|
+-- /connections                --> Neo4j [pendiente]
                                Conexiones entre usuarios
```

El backend Flask actua como capa de abstraccion: el frontend solo conoce la API REST y no sabe que hay cuatro bases de datos diferentes detras. Cada blueprint es responsable de comunicarse con su motor correspondiente: posts_bp usa pymongo para MongoDB, auth_bp/profile_bp/jobs_bp/groups_bp usan el SDK de Supabase para PostgreSQL, notifications_bp y messages_bp usaran Cassandra, y connections_bp usara Neo4j.

9. Conclusion

La eleccion de MongoDB para el modulo de feed social esta justificada por:

- Flexibilidad de esquema: las publicaciones tienen contenido variable (texto, imagen, video, tags) que se representa naturalmente como un documento sin necesidad de tablas auxiliares.
- Alto volumen de escrituras: likes y comentarios son operaciones de alta frecuencia que se benefician del modelo de inserts directos en colecciones separadas sin locks de tabla.
- Aggregation Pipeline: el operador \$lookup permite combinar posts con likes y comentarios en una sola consulta eficiente, incluyendo calculos como likes_count y user_liked.
- Colecciones separadas: el diseno con tres colecciones (posts, likes, comments) evita el limite de 16 MB por documento, facilita los borrados individuales e independiza el crecimiento de cada entidad.
- Integracion con Atlas: MongoDB Atlas proporciona una base de datos cloud gestionada, accesible mediante URI SRV, sin necesidad de infraestructura local.
- Complementariedad con los demas motores: MongoDB no reemplaza a PostgreSQL, Cassandra ni Neo4j sino que se suma a ellos, asumiendo exclusivamente el rol del feed social donde sus características lo hacen mas adecuado.

El resultado es un sistema poliglota donde cada motor asume el dominio que mejor maneja: PostgreSQL garantiza la integridad transaccional de usuarios, empleos y grupos; MongoDB gestiona el contenido social dinamico del feed con flexibilidad de esquema y alto rendimiento de escritura; Cassandra (a implementar) maneja el alto volumen de notificaciones y mensajes con escrituras distribuidas; y Neo4j (a implementar) modelara las conexiones entre usuarios como un grafo para calcular redes de contacto y matching.